

1.(10%) Let the decimal numbers $A = 56$, $B = -81$, $C = 0.65625$

(a) Convert the decimal number C to binary format

$$\begin{array}{r}
 0.65625 \\
 \times \quad 2 \\
 \hline
 1.31250 \rightarrow 1 \\
 \times \quad 2 \\
 \hline
 0.62500 \rightarrow 0 \\
 \times \quad 2 \\
 \hline
 1.25000 \rightarrow 1 \\
 \times \quad 2 \\
 \hline
 0.50000 \rightarrow 0 \\
 \times \quad 2 \\
 \hline
 1.00000 \rightarrow 1
 \end{array}$$

$$A: 0.10101_{(2)}$$

(b) Compute $A + B$ in 8-bit 2's complement, is it overflow? Why or why not?

$$A = 56_{(10)} = 111000$$

$$B = -81_{(10)} = 10101111_{(2)}$$

$$81_{(10)} = 01010001$$

$$\begin{array}{r}
 00111100 \\
 + 10101111 \\
 \hline
 11101011
 \end{array}$$

$$\text{MSB's carry} \oplus (\text{MSB}-1)\text{'s carry} = 0$$

A: 無溢位，因為最高項進位與次高項進位 XOR 為 0

(c) Compute $A - B$ in 8-bit 2's complement, is it overflow? Why or why not?

$$A = 56_{(10)} = 00111000$$

$$B = -81_{(10)} = 10101111$$

$$\begin{array}{r}
 00111000 \\
 - 10101111 \\
 \hline
 10001001
 \end{array}$$

$$\text{MSB's carry} \oplus (\text{MSB}-1)\text{'s carry} = 1 \rightarrow \text{overflow}$$

A: 有溢位，因為最高項進位與次高項進位 XOR 為 1

- 2.(10%) When performing arithmetic addition and subtraction, overflow might occur. Fill in the blanks in the following table of overflow conditions for addition and subtraction.

Operation	Operand A	Operand B	Result indicating overflow
<u>$A+B$</u>	≥ 0	≥ 0	(a) <u>< 0</u>
<u>$A+B$</u>	< 0	< 0	(b) <u>≥ 0</u>
<u>$A-B$</u>	≥ 0	< 0	(c) <u>≤ 0</u>
<u>$A-B$</u>	< 0	≥ 0	(d) <u>≥ 0</u>

- 3.(10%) Prove that the overflow condition can be determined simply by checking to see if the CarryIn to the most significant bit of the result is not the same as the CarryOut of the most significant bit of the result. (Apply true table)

MSB_A	MSB_B	C_{in}	C_o	MSB_{result}	$MSB_{correct}$	Overflow	$C_i \oplus C_o$	Note
0	0	0	0	0	0	No	0	[兩正數相加 結果恆正]
0	0	1	0	1	0	YES	1	
0	1	0	0	1	1	No	0	$ A < B $
0	1	1	1	0	0	No	0	$ A > B $
1	0	0	0	1	1	No	0	$ A > B $
1	0	1	1	0	0	No	0	$ A < B $
1	1	0	1	0	1	YES	1	[兩負數相加 結果為負]
1	1	1	1	1	1	No	0	

Note: 減法運算中 MSB 的 $C_{in} = \overline{Carry}$

- 4.(5%) Assume -40 and 88 are signed 8-bit decimal integers stored in sign-magnitude format. Calculate -40 - 88. Is there overflow, underflow, or neither?

Sign Magnitude: 1 bit represents sign, 7 bit represent number

→ 8 bit: $-(2^7-1) \sim +(2^7-1) = -127 \sim +127$

$-40 - 88 = -128 \cdots$ 超出表示範圍 → underflow

ANS: Underflow

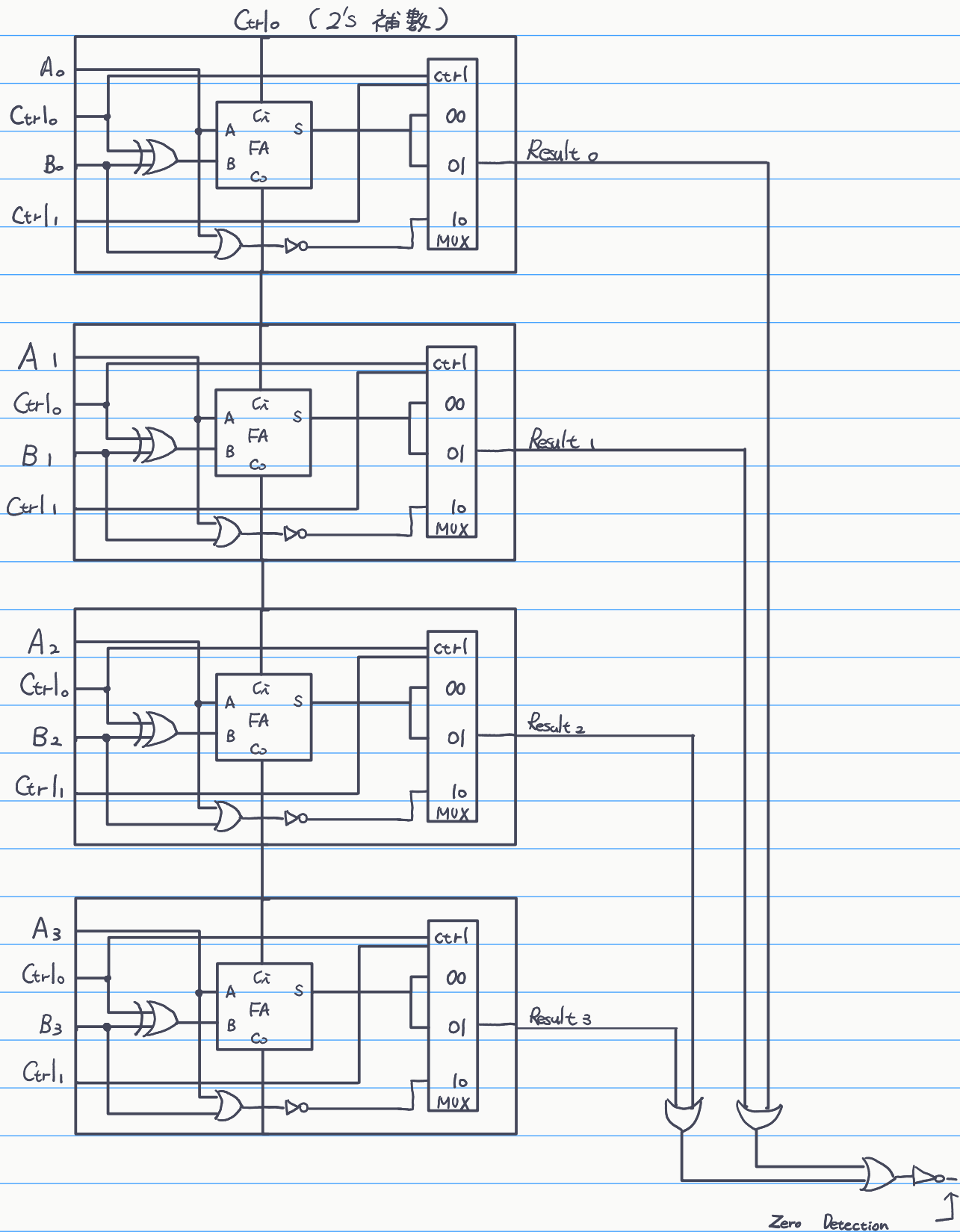
- 5.(5%) What decimal number does the bit pattern 0xC8763000 represent if it is a two's complement integer? An unsigned integer?

$0xC8763000 = 1100\ 1000\ 0111\ 0110\ 0011\ 0000\ 0000\ 0000_{(2)}$

2's complement: $-(1 \times (0011\ 0111\ 1000\ 1001\ 1101\ 0000\ 0000\ 0000)) = -931\ 778\ 560_{\#}$

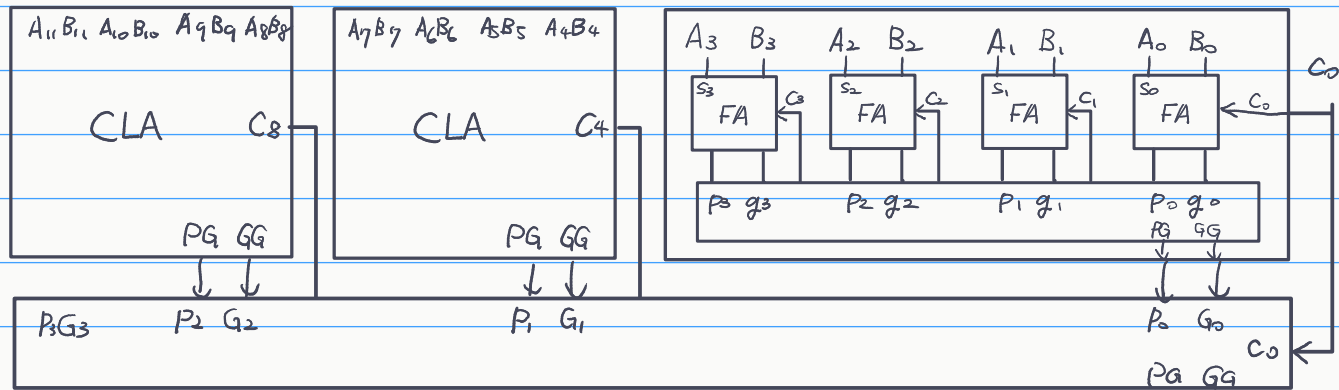
unsigned: $3363\ 1887\ 36_{\#}$

6.(10%) Sketch a 4-bits Adder-Subtractor-Nor-Zero_detection Circuit using 4 Full-Adder (FA) and 4 XOR gates. The input are $(B_3B_2B_1B_0)$, $(A_3A_2A_1A_0)$ and the signal ctrl controls the add/sub operation (ctrl = 00, 01, 10 for Adder/Subtractor/Nor). In your design try to use minimum number of the following basic logic gates (1-bit adders, AND, OR, INV, and XOR, MUX).



7.(10%) Using 4-bit carry-lookahead blocks to design an efficient 12-bit carry-lookahead adder.

(a) Draw the block diagram.



(b) Describe how signals are generated/processed step by step to determine the last Carryout (CarryOut11, C_{in}12) in the following example:

x: 0001 1010 0011

y: 1110 0101 1110

$$\begin{aligned} C_4 &= G_3 + P_3 \cdot (G_2 + P_2 \cdot (G_1 + P_1 \cdot C_0)) \\ &= 0 + 1 \cdot (0 + 1 \cdot (1 + 1 \cdot 0)) \\ &= 1 \end{aligned}$$

$$\begin{aligned} C_8 &= G_7 + P_7 \cdot (G_6 + P_6 \cdot (G_5 + P_5 \cdot C_4)) \\ &= 0 + 1 \cdot (0 + 1 \cdot (0 + 1 \cdot 1)) \\ &= 1 \end{aligned}$$

$$\begin{aligned} C_{12} &= G_{11} + P_{11} \cdot (G_{10} + P_{10} \cdot (G_9 + P_9 \cdot C_8)) \\ &= 0 + 1 \cdot (0 + 1 \cdot (0 + 1 \cdot 1)) \\ &= 1 \end{aligned}$$

ANS: 由以上步驟可以推導出 $C_{12} = 1$

8.(10%) Booth's algorithm is an elegant approach to multiply signed numbers. It starts with the observation that with the ability to both add and subtract there are multiple ways to compute a product. The key to Booth's insight is in his classifying groups of bits into the beginning, the middle, or the end of a run of 1s. Is Booth's algorithm always better? Why does the Booth's algorithm work for multiplication of two's complement signed integers?

(1) 設 multiplier 為 01010101 0

a. 一般乘法器：做四次加法

b. Booth 演算法：做 4 次 $+M$ ，4 次 $-M$

ANS: 由此可知 只要 multiplier 出現越多 01, Booth 的效果就越糟

(2) ANS: 由於 2 補數的 MSB 表示正負號，

對於乘數為正數來說：最後一次會在 $MSB + M$ ，設定正號

對於乘數為負數來說：最後一次會在 $MSB - M$ ，設定負號

使結果符合正負

9.(10%) Using a table similar to that shown in below:

(a) calculate the product of the decimal unsigned 8- bit integers 62 and 35 using the hardware described in Figure 1. You should show the contents of each register on each step.

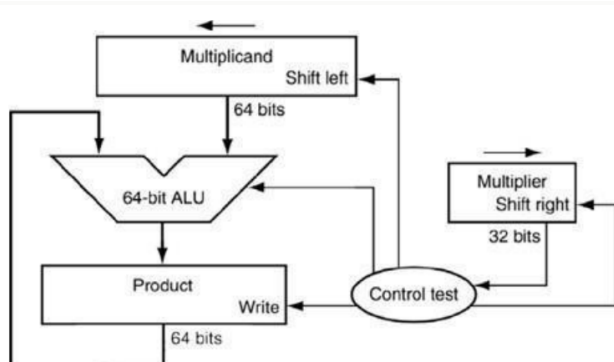
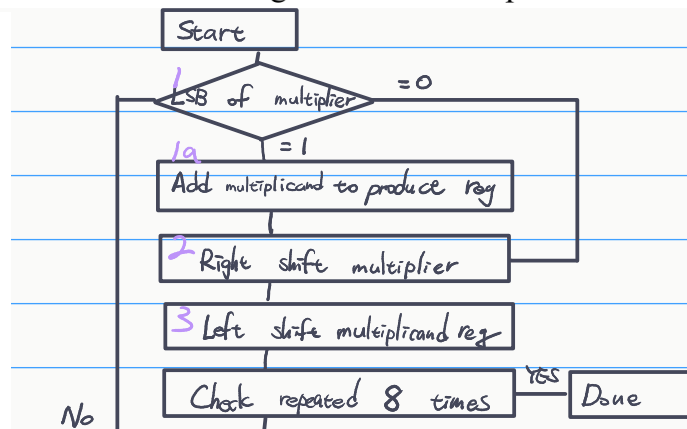


Figure 1. Multiplication hardware



Iteration	step	Multiplier	Multiplicand (M)	product
0	Start, 1	0010 0011	0000 0000 0011 1110	0000 0000 0000 0000
1	1a	0010 0011	0000 0000 0011 1110	0000 0000 0011 1110
	2	0001 0001	0000 0000 0011 1110	0000 0000 0011 1110
	3, 1	0001 0001	0000 0000 0111 1100	0000 0000 0011 1110
2	1a	0001 0001	0000 0000 0111 1100	0000 0000 1011 1010
	2	0000 1000	0000 0000 0111 1100	0000 0000 1011 1010
	3, 1	0000 1000	0000 0000 1111 0000	0000 0000 1011 1010
3	2	0000 0100	0000 0000 1111 0000	0000 0000 1011 1010
	3, 1	0000 0100	0000 0001 1111 0000	0000 0000 1011 1010
4	2	0000 0010	0000 0001 1111 0000	0000 0000 1011 1010
	3, 1	0000 0010	0000 0011 1110 0000	0000 0000 1011 1010
5	2	0000 0001	0000 0011 1110 0000	0000 0000 1011 1010
	3, 1	0000 0001	0000 0111 1100 0000	0000 0000 1011 1010
6	1a	0000 0001	0000 0111 1100 0000	0000 1000 0111 1010
	2	0000 0000	0000 0111 1100 0000	0000 1000 0111 1010
	3, 1	0000 0000	0000 1111 1000 0000	0000 1000 0111 1010
7	2	0000 0000	0000 1111 1000 0000	0000 1000 0111 1010
	3, 1	0000 0000	0001 1111 0000 0000	0000 1000 0111 1010
8	2	0000 0000	0001 1111 0000 0000	0000 1000 0111 1010
	3, 0	0000 0000	0011 1110 0000 0000	0000 1000 0111 1010

(b) In which iterations did the hardware perform an addition operation? How did you determine when to add and when to skip?

ANS: (1) 第 1、2、6 次迭代

(2) 通過檢查 Multiplier 的 LSB, 如果為 1 即做加法, 為 0 則跳過

10.(10%) Please calculate 149 divided by 28 using the hardware described in Figure 5. (Assume 149 and 28 are decimal numbers). You should show the contents of each register on each step. Assume A and B are unsigned 8-bit integers. You will want to think hard about this, do an experiment or two, or else go to the web to figure out how to make this work correctly. (Hint: one possible solution involves using the fact that Figure 5. implies the remainder register can be shifted either direction.)

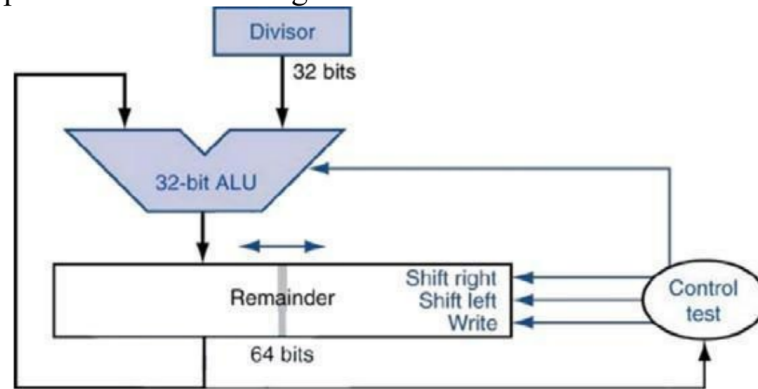
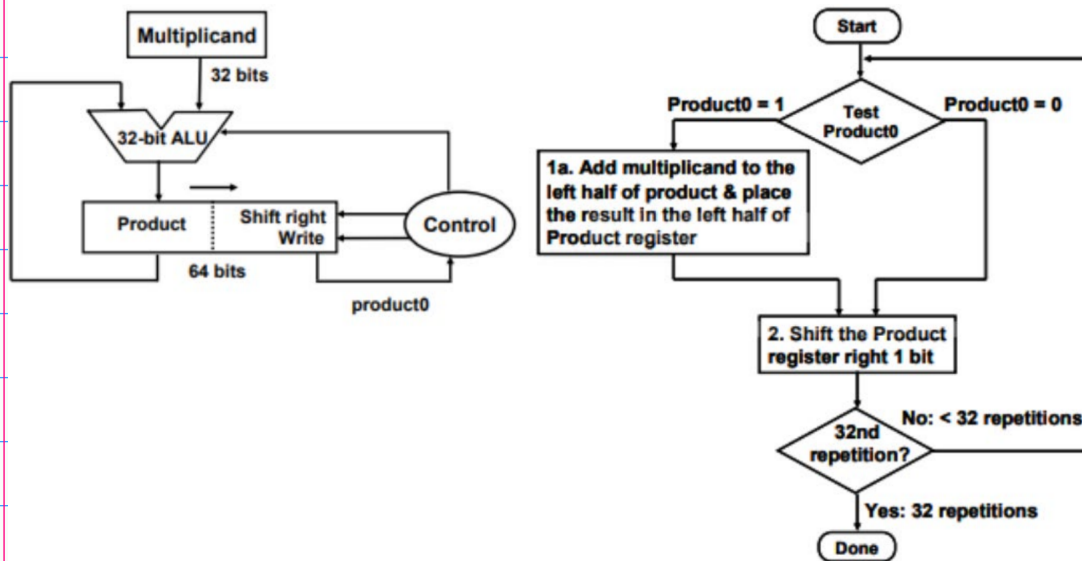


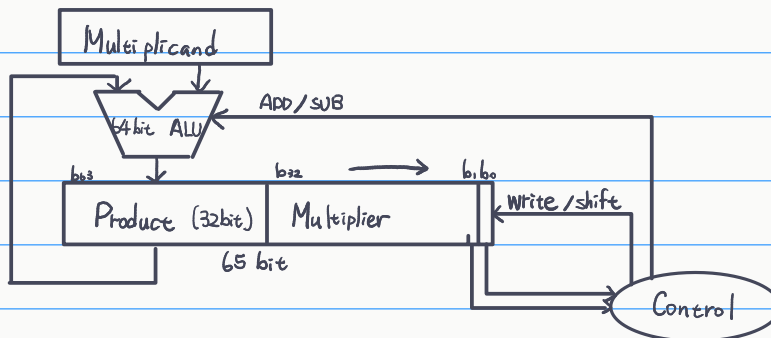
Figure 5. Division hardware

Iteration	Step	Remainder(R)	Divisor
0	Initial Values, 1	0000 0001 0010 1010	0001 1100
1	2: Sub divisor from the left part	1110 0101 0010 1010	0001 1100
	3b: Add divisor left, shift left, see LSB 0	0000 0010 0101 0100	0001 1100
2	2	1110 0110 0101 0100	0001 1100
	3b	0000 0100 1010 1000	0001 1100
3	2	1110 1000 1010 1000	0001 1100
	3b	0000 1001 0101 0000	0001 1100
4	2	1110 1101 0101 0000	0001 1100
	3b	0001 0010 1010 0000	0001 1100
5	2	1111 0101 1010 0000	0001 1100
	3b	0010 0101 0100 0000	0001 1100
6	2	0000 1001 0100 0000	0001 1100
	3a	0001 0010 1000 0001	0001 1100
7	2	1111 0110 1000 0001	0001 1100
	3a	0010 0101 0000 0010	0001 1100
8	2	0000 1001 0000 0010	0001 1100
	3a	0001 0010 0000 0101	0001 1100
Done	D: Shift left half right 1 bit	0000 1001 0000 0101	0001 1100

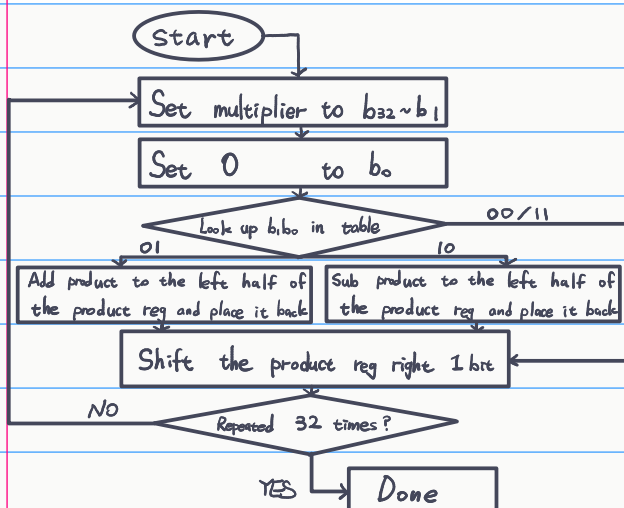
11. (10%) Given the hardware and the flow chart of a multiplication algorithm.



(a) Modify the hardware as less as possible for Booth's multiplication algorithm. Draw the modified hardware.



(b) Based on the modified hardware designed in (a), redraw the flow chart for Booth's multiplication algorithm.



(c) Describe the characteristics and advantages of Booth's algorithm.

ANS: 此演算法可以在少量的硬體下執行負數乘法，
且平均速度較一般乘法器快